

Arquitetura do Sistema - Projeto MaaS

Carteira Multimodal de Mobilidade Urbana com Créditos Integrados e Cashback

Objetivo: criar a arquitetura do sistema para servir como base de implementação na próxima aula. Contexto: usuários urbanos enfrentam dificuldade para gerenciar diferentes aplicativos, saldos, formas de pagamento e créditos de mobilidade. A solução integra transporte público, apps de corrida, bicicleta, patinete e outros modais em uma única carteira digital.

1. Visão Geral da Solução

A plataforma MaaS Wallet será um aplicativo de mobilidade urbana multimodal que permite ao usuário planejar deslocamentos, comparar modais, administrar créditos de transporte e receber cashback por escolhas de mobilidade. A arquitetura considera transição geracional por meio de modo simples, comunicação clara, histórico de transações e notificações.

2. Funcionalidades Principais

2.1 Aplicativo do Usuário

- Cadastro e login; preferências de mobilidade; consulta de saldo; recarga via Pix/cartão/voucher; planejamento de rotas multimodais; comparação por tempo, custo e cashback; pagamento de viagens; extrato; notificações.

2.2 Administração e Operação

- Gestão de parceiros e modais; configuração de cashback; monitoramento de transações; falhas de integração; limites e bloqueios; dashboards operacionais.

2.3 Integração com Parceiros

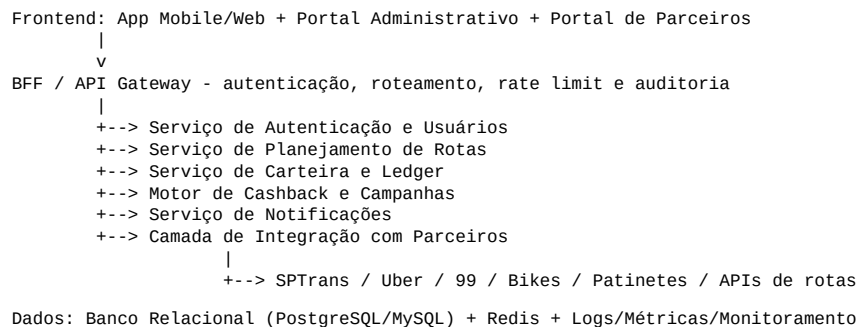
- Catálogo de serviços; cotação; criação de viagem/reserva; confirmação de uso; registro de cobrança, estorno e cashback; webhooks de status.

3. Tipos de Usuários e Permissões

Tipo	Descrição	Permissões
Passageiro	Usuário final	Conta, recarga, rota, pagamento, saldo, cashback
Administrador MaaS	Equipe da plataforma	Usuários, parceiros, regras, limites, relatórios e auditoria
Parceiro de Mobilidade	Operadoras e provedores	Status de viagens, transações próprias e conciliação
Operador de Atendimento	Suporte	Consulta de transações e abertura de chamados, sem ajuste direto de saldo
Gestor Corporativo	Empresa cliente	Distribuição de créditos e relatórios agregados

Regras de autorização: o passageiro acessa apenas a própria conta; parceiro acessa apenas suas transações; ajustes financeiros exigem aprovação administrativa; relatórios corporativos devem respeitar LGPD.

4. Diagrama de Arquitetura em Camadas



Descrição: a solução separa frontend, backend, integrações e dados. O backend é modular, com domínio financeiro isolado no serviço de carteira e ledger, reduzindo risco de inconsistência e facilitando auditoria.

5. Entidades Principais e Relacionamentos

Usuario 1---N Carteira
Carteira 1---N Transacao
Usuario 1---N Viagem
Viagem N---1 Modal
Viagem N---1 Parceiro
Viagem 1---N Cashback
Parceiro 1---N Modal
Parceiro 1---N ProdutoMobilidade
CampanhaCashback 1---N Cashback
GestorCorporativo 1---N VoucherCredito
VoucherCredito N---1 Carteira

- Usuario: dados cadastrais, tipo e status.
- Carteira: saldo disponível, saldo cashback e status.
- Transacao: recarga, débito, cashback, estorno ou ajuste.
- Parceiro: empresa integrada e status da integração.
- Viagem: cotação, reserva, andamento, conclusão ou cancelamento.
- CampanhaCashback: regra, percentual, período, modal elegível e teto de uso.

6. Endpoints Principais da API REST

Grupo	Método/Endpoint	Descrição
Auth	POST /api/v1/auth/register	Cadastro de usuário
Auth	POST /api/v1/auth/login	Login
Usuário	GET /api/v1/users/me	Consultar perfil
Carteira	GET /api/v1/wallet	Consultar saldo
Carteira	POST /api/v1/wallet/recharge	Solicitar recarga
Carteira	GET /api/v1/wallet/transactions	Consultar extrato
Rotas	POST /api/v1/routes/search	Buscar opções multimodais
Viagens	POST /api/v1/trips/quote	Gerar cotação
Viagens	POST /api/v1/trips	Criar viagem/reserva
Cashback	GET /api/v1/rewards/campaigns	Listar campanhas
Admin	POST /api/v1/admin/partners	Cadastrar parceiro
Parceiros	POST /api/v1/partners/webhooks/trip-status	Receber status de viagem

7. Tecnologias Sugeridas

- Frontend: Flutter ou React Native para app mobile; React/Angular/Vue para portal administrativo.
- Backend: Spring Boot, NestJS ou FastAPI, com arquitetura modular por domínio.
- Banco de dados: PostgreSQL ou MySQL para transações; Redis para cache; OpenSearch/ELK para logs.
- Integrações: REST APIs, webhooks, gateway de pagamento Pix/cartão e API de mapas/rotas.
- Segurança: OAuth2/JWT, HTTPS, RBAC, criptografia de dados sensíveis, logs de auditoria e LGPD.
- DevOps: Docker, CI/CD, ambientes de homologação e produção, métricas e alertas.

8. Regras de Negócio Importantes

- Toda movimentação financeira deve gerar registro imutável no ledger.
- Saldo exibido deve considerar transações confirmadas, pendentes e bloqueadas.
- Cashback só é creditado após confirmação da viagem ou consumo.
- Campanhas devem ter vigência, teto de uso e modal elegível.

- Usuário não inicia viagem sem saldo suficiente, exceto quando o modal aceita cobrança externa.
- Estornos mantêm rastreabilidade da transação original.
- Webhooks devem ser idempotentes para evitar duplicidade de cobrança.
- Parceiros acessam somente transações próprias.
- Dados pessoais devem respeitar LGPD e minimização de dados.

9. Fluxos Principais

Cadastro e recarga: usuário cria conta, solicita recarga, gateway confirma pagamento, carteira credita saldo e ledger registra a transação.

Planejamento e pagamento: usuário informa origem/destino, sistema calcula rotas, consulta parceiros, mostra opções com custo/tempo/cashback, usuário escolhe, carteira reserva ou debita saldo, parceiro confirma viagem.

Conclusão e cashback: parceiro envia webhook de viagem concluída, sistema verifica idempotência, confirma débito final, calcula cashback e registra recompensa no ledger.

10. Estratégia de Implementação para a Próxima Aula

- Implementar MVP com login, carteira simulada, recarga simulada via Pix, rotas mockadas, parceiros mockados, cashback simples por percentual, extrato e dashboard básico.
- Essa estratégia reduz dependência de integrações reais e permite validar arquitetura, regras financeiras e fluxos de usuário.

11. Prompts Utilizados para Criar a Arquitetura

Ferramenta utilizada: ChatGPT - apoio à estruturação de arquitetura, regras de negócio, entidades, endpoints e fluxos.

Prompt 1: Crie a arquitetura de um sistema MaaS para uma startup de mobilidade urbana que administra créditos de transporte multimodal em uma carteira única. A solução deve integrar transporte público, Uber, 99, bicicletas e patinetes, permitindo pagamento, planejamento de rotas e cashback. Considere regras de negócio, segurança, LGPD e integração com parceiros externos.

Prompt 2: Defina uma arquitetura em camadas para o sistema MaaS, contendo frontend, backend, banco de dados, camada de integração, autenticação, carteira digital, motor de cashback e APIs externas. Use Mermaid para representar o diagrama.

Prompt 3: Modele as principais entidades e relacionamentos do sistema, incluindo usuário, carteira, transação, parceiro, modal, produto de mobilidade, viagem, cashback e campanha. Crie também os principais endpoints REST para implementação.

Prompt 4: Liste as regras de negócio críticas para uma carteira de créditos multimodal com cashback e descreva os fluxos principais de cadastro, recarga, planejamento de viagem, pagamento, confirmação por parceiro e crédito de cashback.

12. Conclusão

A arquitetura proposta fornece uma base clara para implementação da plataforma MaaS Wallet. Como experiência profissional aplicada ao projeto, foram priorizados modularidade, observabilidade, segurança, ledger financeiro, idempotência em webhooks, RBAC, logs de auditoria e separação entre carteira, rotas e recompensas.